LABORATORIO DI PROGRAMMAZIONE I

Lezione 4

Programmazione orientata ad oggetti

Laura Nenzi

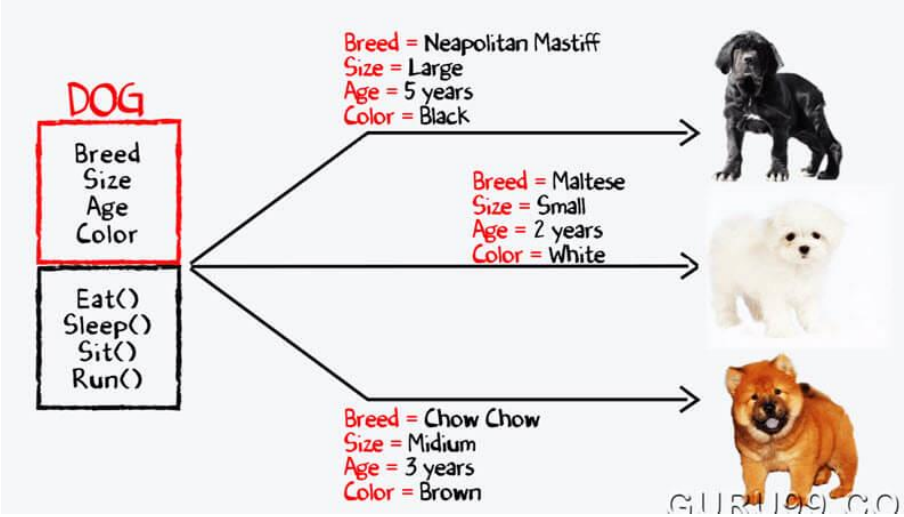
Programmazione Orientata agli oggetti (OPP)

- Pensiamo a una lampadina
  - C’è il concetto di lampadina che ha alcune caratteristiche:
    - Cose che si possono fare (accendere/spegnere)
    - Stato interno (se è accesa/spenta)
  - Una specifica lampadina rispetta il “template” della lampadina ma ha alcune caratteristiche fissate
  - Due istanze del concetto di lampadina (i.e., due lampadine) avranno stato interno diverso ma possiamo svolgere le stesse operazioni su di loro

Programmazione Orientata agli oggetti (OPP)

- La programmazione orientata agli oggetti è un modo di strutturare i programmi aggregando in “oggetti”:
  - Il comportamento
  - Le proprietà / stati
- Esempio: un oggetto può rappresentare una lampadina:
  - Proprietà: acceso/spento, tipo (led/incandescenza), Watt
  - Comportamento: “accendi”, “svita”, etc.

Programmazione Orientata agli oggetti (OPP)



Programmazione Orientata agli oggetti (OPP)

E’ un paradigma di programmazione

Permette di ragionare in termini di entità e non solo di funzioni.

Gli ***oggetti*** sono definiti con le ***classi***

Una classe è uno schema che permette di descrivere un’entità con le caratteristiche desiderate.

Programmazione Orientata agli oggetti (OPP)

Negli oggetti/classi:

- le funzioni si chiamano ***metodi***
- le variabili si chiamano ***attributi***

Una volta inizializzati diventano ***istanze***
→ si parla infatti di *istanziare* una classe

Programmazione Orientata agli oggetti (OPP)

- Si consideri un **classe** “Cat”:
  - Rappresenta il concetto generico di un gatto
  - Dobbiamo indicare che attributi ha (e.g., nome ed età).
    Questi sono gli **attributi** dell’istanza
  - Indichiamo che comportamenti può avere (e.g., miagolare).
    Questi sono i **metodi**
- Un gatto specifico può essere Tom, che avrà: età==13 e nome==“Tom”
  - Essendo una istanza della classe “Cat” sappiamo che potrà miagolare

Programmazione Orientata agli oggetti (OPP)

Inizializzazione / istanziamento

Classe per l’oggetto Persona

```
- nome
- saluta()
    print 'Ciao!'
```

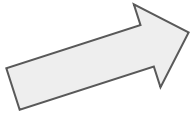
Istanza per l’oggetto persona

```
- nome = Mario
- saluta()
    print 'Ciao!'
```

Programmazione Orientata agli oggetti (OPP)

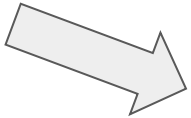
Classe per l’oggetto Persona

```
- nome
- saluta()
     print 'Ciao!'
```



Istanza per l’oggetto persona

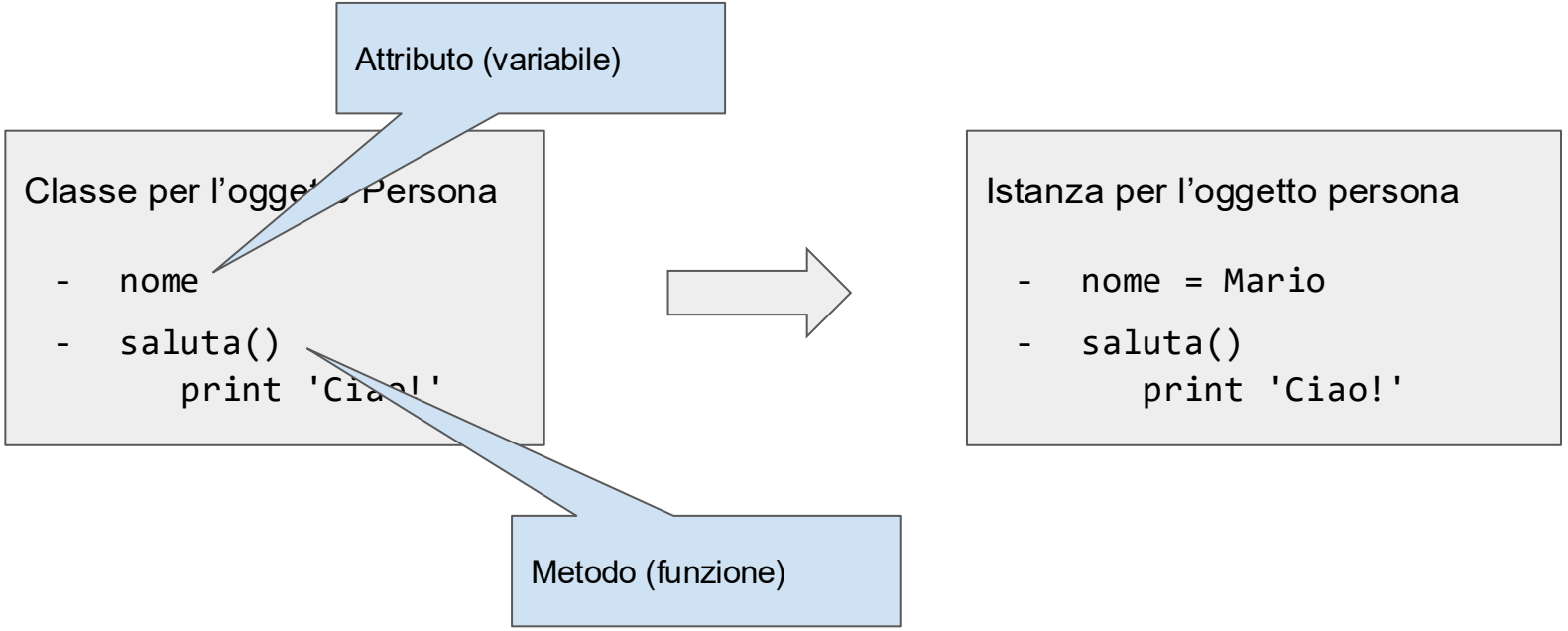
```
- nome = Mario
- saluta()
     print 'Ciao!'
```



Istanza per l’oggetto persona

```
- nome = Lucia
- saluta()
     print 'Ciao!'
```

Programmazione Orientata agli oggetti (OPP)



Programmazione Orientata agli oggetti (OPP)

Gli oggetti si usano principalmente perchè:

- Permettono di rappresentare bene delle gerarchie (e sfruttare le caratteristiche in comune)

- Una volta istanziati, permettono di mantenere facilmente lo *stato* (senza diventare matti con strutture dati di appoggio)

Classi in Python

- In Python le classi sono un modo di definire strutture dati (e operazioni su di esse) da parte dell’utente
- Una classe fornisce una struttura (“blueprint”) per qualcosa che deve essere definito, senza però fornire il contenuto (i.e., i valori degli attributi specifici)
- Per la classe “Cat” specificherà che un gatto deve:
    - Avere un nome e una età (ma non dirà che valori sono questi)
    - Che un gatto può miagolare

In Python tutto è un oggetto

```
>>> my_string_2 = 'corso di laboratorio di programmazione'
>>> dir(my_string_2)
['__add__', '__class__', '__contains__', '__delattr__', '__dir__', '__doc__', '__eq__', '__for
mat__', '__ge__', '__getattribute__', '__getitem__', '__getnewargs__', '__gt__', '__hash__', '
__init__', '__init_subclass__', '__iter__', '__le__', '__len__', '__lt__', '__mod__', '__mul__
', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__rmod__', '__rmul__', '__
setattr__', '__sizeof__', '__str__', '__subclasshook__', 'capitalize', 'casefold', 'center', '
count', 'encode', 'endswith', 'expandtabs', 'find', 'format', 'format_map', 'index', 'isalnum'
, 'isalpha', 'isascii', 'isdecimal', 'isdigit', 'isidentifier', 'islower', 'isnumeric', 'ispri
ntable', 'isspace', 'istitle', 'isupper', 'join', 'ljust', 'lower', 'lstrip', 'maketrans', 'pa
rtition', 'replace', 'rfind', 'rindex', 'rjust', 'rpartition', 'rsplit', 'rstrip', 'split', 's
plitlines', 'startswith', 'strip', 'swapcase', 'title', 'translate', 'upper', 'zfill']
>>> my_string_2.title()
'Corso Di Laboratorio Di Programmazione'
```

La funzione dir() ritorna una lista con tutti gli attributi e i metodi di un oggetto

In Python tutto è un oggetto

examples.py

```python
my_string = 'a,b,c'
print(my_string)
print(my_string.split(','))
print(my_string)
```

```
> python examples.py
a,b,c
['a', 'b', 'c']
a,b,c
```

examples.py

```python
my_list = [1,2,3,4]
print(my_list)
print(my_list.reverse())
print(my_list)
```

```
> python examples.py
[1, 2, 3, 4]
None
[4, 3, 2, 1]
```

Oggetti con operazioni in-place e non

examples.py

```
my_string = 'a,b,c'
print(my_string)
print(my_string.split(','))
print(my_string)
```

Questa è un'operazione (funzione, metodo) che quando viene eseguita torna il risultato

```
> python examples.py
a,b,c
['a', 'b', 'c']
a,b,c
```

examples.py

```
my_list = [1,2,3,4]
print(my_list)
print(my_list.reverse())
print(my_list)
```

Questa è un'operazione (funzione, metodo) **in-place** che quando viene eseguita modifica l'oggetto, non torna niente!

```
> python examples.py
[1, 2, 3, 4]
None
[4, 3, 2, 1]
```

Definire classi

- Per definire una classe in Python usiamo la parola chiave **class**:
  - **class NomeClasse:**
  - Tutta il codice contenuto all'interno sarà parte della classe…
  - …ma che codice dobbiamo mettere?
  - Se vogliamo solo dire che una classe esiste, senza dire nulla di attributi e metodi (proprietà e comportamenti) possiamo semplicemente usare la keyword "pass"
  - **class Cat:**
    pass

Definire oggetti

objects.py

```python
class Person():
    pass

person = Person()
print(person)
```

```
> python objects.py
<__main__.Person object at 0x7ff378a93fa0>
>
```

Definire oggetti

objects.py

```
class Person():
    pass

person = Person()
print(person)
```

“pass” è l’istruzione nulla, serve per avere un blocco vuoto

Questa operazione *istanzia* una classe di oggetto.

Ovvero, da una generica definizione di un oggetto ne creo uno specifico.

```
> python objects.py
<__main__.Person object at 0x7ff378a93fa0>
>
```

Creazione di una istanza

- Così facendo abbiamo creato una classe “Cat”…
- …ma non abbiamo creato una istanza specifica di “Cat”
- La classe è un “blueprint” (i.e., una idea astratta)…
- …dobbiamo ora creare una istanza specifica di “Cat”
- Però per il momento non abbiamo definito nessun comportamento specifico o nessuna proprietà…
- …come possiamo fare in Python?

Attributi di istanza

- Tutte le classi sono usate per creare oggetti e tutti gli oggetti contengono delle caratteristiche/proprietà/attributi
- Questi possono venire impostati al momento della creazione dell'oggetto (i.e., quando "riempiamo" il blueprint in caso concreto)
- Al momento della creazione viene chiamato il metodo **__ init(self, …)__**
    - Questo metodo è utilizzato per inizializzare gli attributi di un oggetto al momento della creazione
    - Questo metodo (o equivalente in altri linguaggi) è chiamato il costruttore

Il metodo __init__

- Il metodo __init__ ha come argomenti:
  - “self”, che indica l’oggetto stesso (equivalente a “this” in linguaggi come Java)
  - Se fossimo in C “self” sarebbe un puntatore alla struttura stessa, in modo da poter dire “voglio accedere al campo x di questa struttura”
  - Gli argomenti dopo il primo possono essere aggiunti per inizializzare lo stato dell’oggetto (e.g., impostare gli attributi)

Il metodo __init__

- Possiamo aggiungere due argomenti al costruttore di Cat, name e age.
- Per salvarli come attributi possiamo usare
  self.nome_attributo = …
  Se non era già presente viene creato sul momento
- Una volta creata l'oggetto e assegnato a una variabile possiamo accedere agli stessi attributi allo stesso modo anche fuori dal costruttore:
  - e.g.,
    x = Cat("Tom", 13)
    print(x.name)

Il metodo __init__

```python
class Person():

    def __init__(self, name, surname):

        # Set name and surname
        self.name    = name
        self.surname = surname

person = Person('Mario', 'Rossi')
print(person)
print(person.name)
print(person.surname)
```

```
> python objects.py
<__main__.Person object at 0x7f8a75ac0fa0>
Mario
Rossi
>
```

Il metodo __init__

la funzione _init_ (costruttore) è quella che è responsabile di inizializzare l'oggetto, i.e. istanzia gli attributi.
Se non c'è viene usata quella di default che non fa nulla.

```
class Person():

    def __init__(self, name, surname):

        # Set name and surname
        self.name    = na
        self.surname = surn

person = Person('Mario', 'Rossi
print(person)
print(person.name)
print(person.surname)
```

```
> python objects.py
<__main__.Person object at 0x7f8a75ac0fa0>
Mario
Rossi
>
```

"self" vuol dire "me stesso", "me *istanza* di classe". E' obbligatorio come parametro in tutti i metodi degli oggetti (salvo casi particolari)

È prassi che i parametri di __init__ abbiano gli stessi nomi degli attributi. Se metto dei valori di default diventano opzionali.
Buona prassi è inizializzare tutti gli attributi di un oggetto.

Attributi di classe

- In alcuni casi ci sono caratteristiche comuni a tutte le istanze di una classe
- In questo caso ha senso che siano definite a livello di classe e non di singola istanza
- Queste variabili sono definite dentro la classe ma fuori dal metodo __init__
- Saranno accessibili usando **NomeClasse.NomeAttributo**…
- …ma anche come se fossero normali attributi di una istanza

```
class Persona:
    nazione = Italia
    def __init__(self, nome, cognome):
        self.nome = nome
        self.cognome = cognome
persona = Persona('Mario', 'Rossi')
print(persona.nazione)
```

Assegnamento: attributi di classe e di istanza

- Possiamo modificare gli attributi di una istanza senza problemi (chiaramente cambieranno solo per quella istanza)
- E per gli attributi di classe?
  - Se li modifichiamo accedendo tramite una istanza verranno modificati solo per quella istanza (che da quel momento in poi avrà una sua “copia locale”)
  - Se li modifichiamo accedendo tramite il nome della classe li modifichiamo per tutte le istanze

```
person = Person("Mario", "Rossi")
print(person)
person.surname = "Verdi"
print(person)
```

```
(.conda) (base) laurane
Person "Mario Rossi"
Person "Mario Verdi"
```

Metodi di una classe

- Per definire il comportamento di tutte le istanze di una particolare classe possiamo definire dei metodi

- I metodi di istanza sono definiti, come il metodo __init__, all’interno di una classe

- Il primo argomento è “self”, ma possono avere quanti argomenti vogliamo

Metodi di una classe

```python
class Person():

    def __init__(self, name, surname):

        # Set name and surname
        self.name    = name
        self.surname = surname

    def __str__(self):
        return 'Person "{} {}"'.format(self.name, self.surname)

person = Person('Mario', 'Rossi')
print(person)
```

```
> python objects.py
Person "Mario Rossi"
>
```

Metodi di una classe

```python
class Person():

    def __init__(self, name, surname):

        # Set name and surname
        self.name    = name
        self.surname = surname

    def __str__(self):
        return 'Person "{} {}"'.format(self.name, self.surname)

person = Person('Mario', 'Rossi')
print(person)
```

Funzione ad uso interno. E' responsabile della rappresentazione in formato stringa dell'oggetto.

```
> python objects.py
Person "Mario Rossi"
>
```

```python
# Import the random module
import random

class Person():

    def __init__(self, name, surname):

        # Set name and surname
        self.name      = name
        self.surname   = surname

    def __str__(self):
        return 'Person "{} {}"'.format(self.name, self.surname)

    def say_hi(self):

        # Generate a random number between 0, 1 and 2.
        random_number = random.randint(0,2)

        # Choose a random greeting
        if random_number == 0:
            print('Hello, I am {} {}.'.format(self.name, self.surname))
        elif random_number == 1:
            print('Hi, I am {}!'.format(self.name))
        elif random_number == 2:
            print('Yo bro! {} here!'.format(self.name))

person = Person('Mario', 'Rossi')
person.say_hi()
```

```
> python objects.py
Hello, I am Mario Rossi.
>
```

```
> python objects.py
Hi, I am Mario!
>
```

```
> python objects.py
Yo bro! Mario here!
>
```

Funzione (metodo) dell'oggetto. Anche chiamata interfaccia. Sono le funzioni che verranno testate per l'esame!

```python
# Import the random module
import random

class Person():

    def __init__(self, name, surname):

        # Set name and surname
        self.name       = name
        self.surname    = surn

    def __str__(self):
        return 'Person "{} {}"'.format(self.name, self.surname)

    def say_hi(self):

        # Generate a random number between 0, 1 and 2.
        random_number = random.randint(0,2)

        # Choose a random greeting
        if random_number == 0:
            print('Hello, I am {} {}.'.format(self.name, self.surname))
        elif random_number == 1:
            print('Hi, I am {}!'.format(self.name))
        elif random_number == 2:
            print('Yo bro! {} here!'.format(self.name))

person = Person('Mario', 'Rossi')
person.say_hi()
```

```
> python objects.py
Hello, I am Mario Rossi.
>
```

```
> python objects.py
Hi, I am Mario!
>
```

```
> python objects.py
Yo bro! Mario here!
>
```

Stile di codice

- Usa un'**indentazione di 4 spazi** e non usare tabulazioni. Le tabulazioni introducono confusione e sarebbe meglio evitarle.
- Suddividi le righe in modo che non superino **79 caratteri**. Questo aiuta gli utenti con display piccoli e rende possibile visualizzare più file di codice affiancati su schermi più grandi.
- Usa righe vuote per separare funzioni e classi, oltre che per separare blocchi di codice più grandi all'interno delle funzioni.
- Quando possibile, inserisci i commenti su una loro riga separata.
- Usa le **docstring**.
- Usa spazi intorno agli operatori e dopo le virgole, ma **non direttamente all'interno delle parentesi**: ad esempio a = f(1, 2) + g(3, 4).

Stile di codice

In Python c'è una convenzione di stile ben precisa:

- notazione **snake_case** (caratteri minuscoli e underscore) per i nomi dei **metodi,** delle **variabili** e delle **istanze** degli oggetti
- notazione **CamelCase** per il nome delle **classi**

Inoltre, doppi underscore prima e dopo il nome di un metodo indicano un metodo ad uso esclusivamente interno (esempio: `__str__`, oppure `__doc__`)

Gli apici valgono sia singoli che doppi, ma conviene usarli singoli per il codice, doppi <u>nelle</u> stringhe, `e.g. mystring = 'Il mio nome è "Mario" e sono una persona'`

Stile di codice

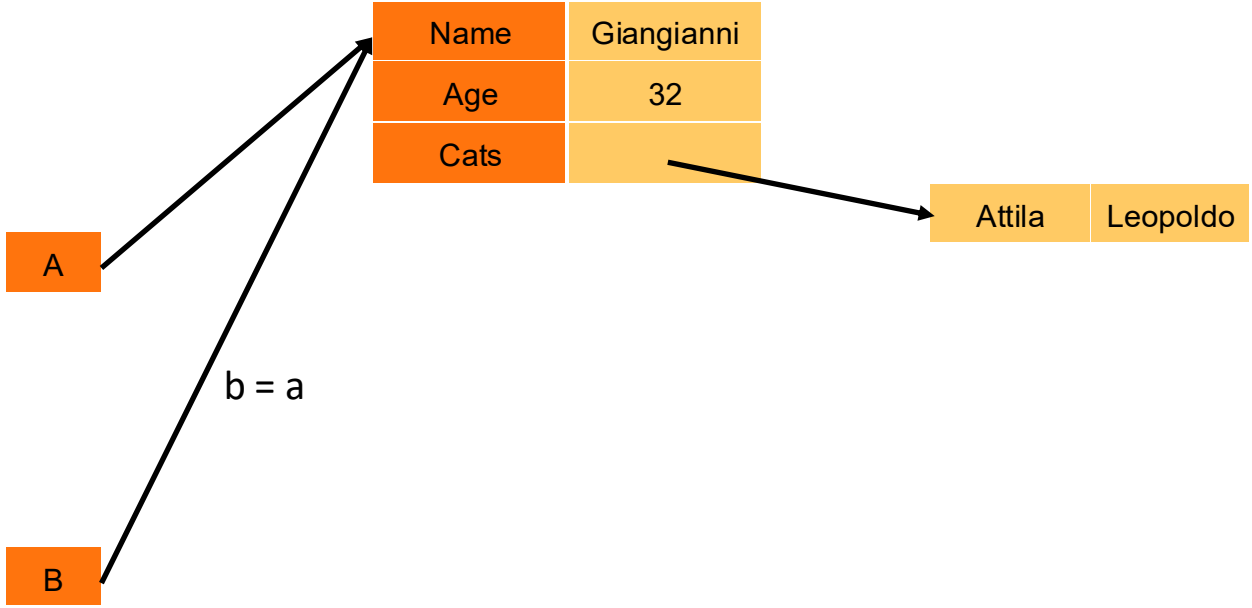
- Usa sempre **self** come nome per il primo argomento di un metodo
- Non utilizzare codifiche complesse se il tuo codice è destinato a essere utilizzato in ambienti internazionali. Il semplice **ASCII** funziona meglio in ogni caso.
- Tutte le convenzioni e molto altro lo trovate nei PEP (Python Enhancement Proposal). Servono come documentazione tecnica per spiegare le motivazioni, i dettagli e gli effetti delle modifiche proposte.
- Il PEP 8 - Style Guide for Python Code: https://peps.python.org/pep-0008/

Copia di oggetti

- Se lavoriamo su interi sappiamo che il seguente codice non modifica x:
  ```
  x = 3
  y = x
  y = 4
  ```
- Cosa succede invece con gli oggetti?
  ```
  x = Person("Mario")
  y = x
  y.name = "Giovanni"
  ```
- Quando facciamo un assegnamento stiamo assegnando un riferimento…
- …e quindi lavoriamo sulla stessa istanza (i.e., c'è un solo oggetto, che non viene copiato)

Copia di oggetti

Copiare riferimenti



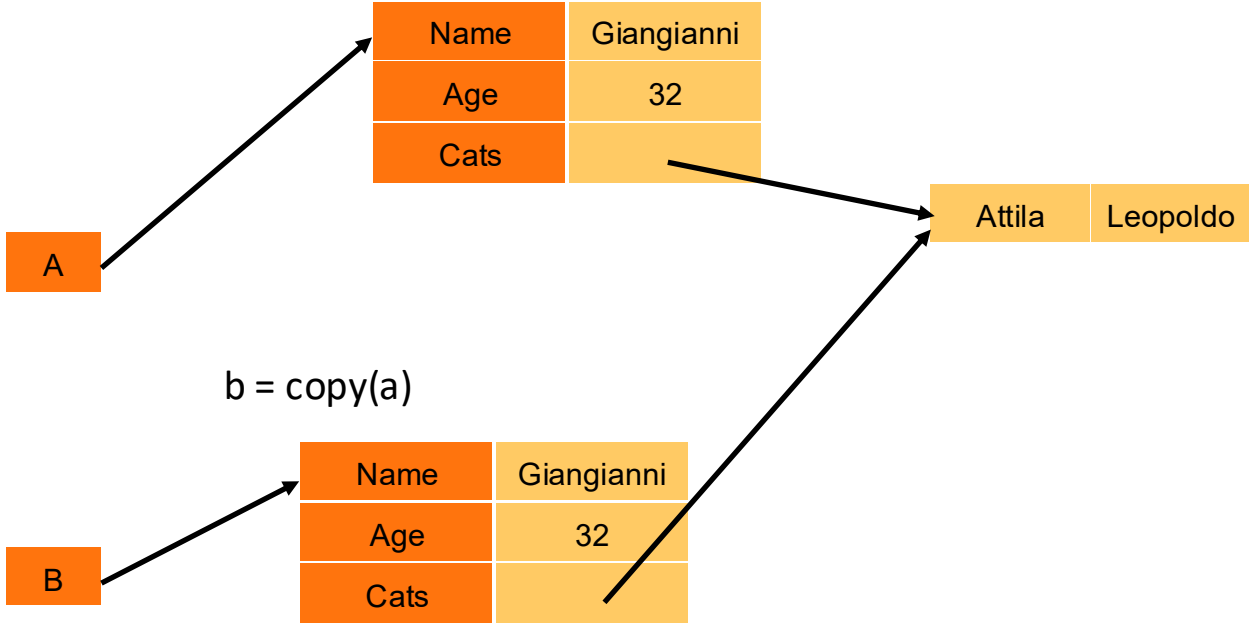
Copia di oggetti

Come risolvere?

- Possiamo pensare di avere un metodo statico per costruire una copia a partire da un originale
- Possiamo usare la libreria “copy”:
  `from copy import copy, deepcopy`
- “*copy*” fornisce una copia “shallow” dell’oggetto: copia tutti gli attributi (facendo assegnamenti) in una nuova istanza dell’oggetto
- “*deepcopy*” fa invece una copia anche di tutti gli attributi, la differenza è si vede uno degli attributi è un oggetto!

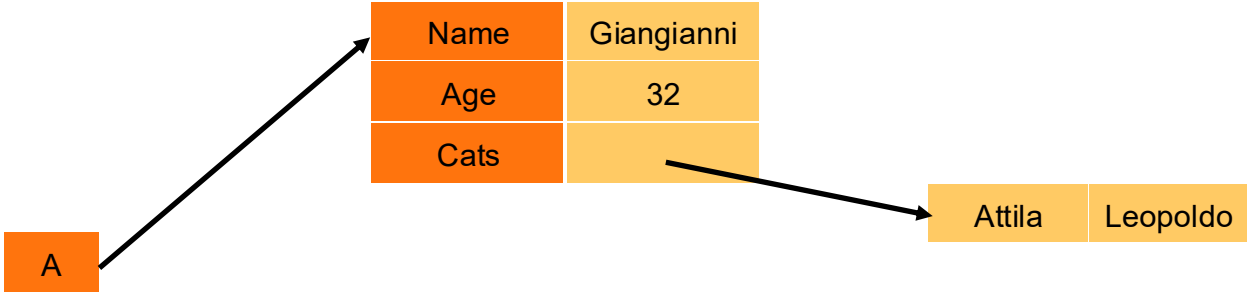
Copia di oggetti

Copy

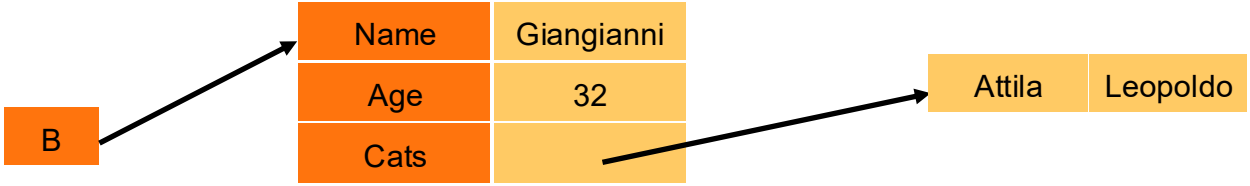


Copia di oggetti

Deepcopy



b = deepcopy(a)



Esempio/Esercizio

- Creare una classe coin che ha come attributo il valore della **faccia**
- E ha due metodi:
  - uno che simula il lancio della moneta e che salva il valore sull'attributo faccia.
  - uno che ritorna il valore dell'attributo faccia

Esercizio 1: **Classe Veicolo**

Scrivete una classe denominata **Veicolo** che disponga di questi attributi dati:

- **modello** (per il modello del veicolo);
- **marca** (per la marca del veicolo);
- **anno** (per l'anno del veicolo).
- **speed** (per la velocità del veicolo)

E di questi metodi:

- __init__ che accetti come argomenti l'anno, il modello, e la marca. Il metodo dovrebbe inoltre assegnare **0** all'attributo dati speed.
- **__str__** che restituisce una stringa con i dettagli del veicolo (marca, modello, anno e velocità)
- **accellerare** che aggiunge **5** all'attributo dati speed ogni volta che viene chiamato.
- **frenare** che sottrae **5** dall'attributo dati speed ogni volta che viene chiamato.
- **get_speed** che restituisce la velocità corrente.

Esercizio 2

Create un oggetto **CSVFile** che rappresenti un file CSV, e che:

1) venga inizializzato sul nome del file csv, e
2) abbia un attributo “name” che ne contenga il nome
3) abbia un metodo “get_data()” che torni i dati dal file CSV come lista di liste, ad es: `[ ['01-01-2012', '266.0'], ['01-02-2012', '145.9'], ... ]`